

Introduction to Linked Lists

Q. What is a Linked List? Explain its representation in memory with the help of a suitable diagram and structure definition in C.

✦ Definition to Remember

A **Linked List** is a linear, dynamic data structure consisting of a sequence of elements called **nodes**. Unlike arrays, memory is NOT allocated contiguously. Instead, each node contains **data** and a **pointer (link)** to the memory address of the next node, forming a chain.

1. Structure of a Node

A standard linked list node has two components:

1. **Data Field:** Stores the actual information.
2. **Next (Link) Field:** A pointer holding the memory address of the next node.

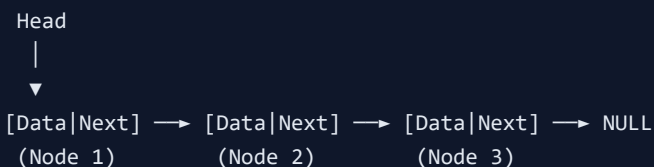
Structure Definition in C:

```
struct Node {  
    int data;           // Data field  
    struct Node *next;  // Pointer to the next node (Self-Referential Structure)  
};
```

2. Representation in Memory

- The list is accessed via a special starting pointer called the **head** (or **start**).
- If the list is empty, **head == NULL**.
- The **last node** in the list has its **next** pointer set to **NULL** to signify the end of the chain.

Diagram:



(Nodes are scattered in memory, connected only by these pointers).

3. Advantages over Arrays

- **Dynamic Memory Allocation:** Size can grow and shrink at runtime; memory is allocated exactly as needed (no wastage).
- **Efficient Insertion/Deletion:** Adding or removing a node doesn't require shifting elements — just updating pointers.

★ Must-Write Points (for 10 marks)

1. A Linked List is a linear, **dynamic** data structure made of **nodes**.
2. Nodes are **not stored in contiguous memory** locations.
3. Each node contains two parts: **Data** and **Next (pointer to next node)**.
4. `struct Node { int data; struct Node *next; };` (Self-Referential Structure).
5. The **head** pointer points to the first node; if empty, **head = NULL**.
6. The last node's pointer is set to **NULL** to indicate the end.
7. Primary advantages: dynamic size and efficient insertion/deletion without shifting elements.

⚡ Quick Recall

**Linked List → Dynamic Size → Nodes (Data + Next Pointer) → Non-contiguous memory → Head points to start
→ Last node points to NULL → struct Node { int data; struct Node *next; }**